

Master Git, GitHub & SSH Reference Guide

Installation Blueprint, Secure Key Cryptography & Cloud Synchronization Matrix

The Core Framework: Git runs locally on your hardware, while GitHub is a cloud-hosted framework for project distribution [cite: 5]. To push and pull code securely without typing personal access tokens constantly, modern workflows utilize SSH cryptographic key pairs. This master reference guide details platform configuration, secure access handshake provisioning, and syncing pipelines.

1. Git Engine Installation & Environment Verification

Target Platform	Instruction & Core Mechanics	Terminal Installation Commands
Windows 11 / 10	Execute via PowerShell. Uses the native Windows Package Manager to grab the official distribution package [cite: 5]. Restart the shell context immediately after completion [cite: 5].	<pre>winget install --id Git.Git -e --source winget</pre>
Linux (Ubuntu / Debian)	Refresh local package indices via Advanced Package Tool (apt) and trigger the core engine installation layout [cite: 5].	<pre>sudo apt update sudo apt install git -y</pre>
macOS Systems	Provision using the Homebrew package framework or trigger Apple command-line compiler tools naturally [cite: 5].	<pre>brew install git</pre>
Verification Check	Launch a brand new terminal instance to force a profile path reload, then execute a basic environmental smoke test [cite: 5].	<pre>git --version # Returns: git version 2.X.X</pre>

2. GitHub Identity & Cross-Platform Configuration

Deployment Stage	Instruction & Core Mechanics	Execution Parameters
Local Global Identity	Map your exact name and registration email credentials to your machine's global configuration map to tag authorship strings accurately [cite: 5].	<pre>git config --global user.name "Your Name" git config --global user.email "email@example.com"</pre>
Cross-Platform Line Endings	CRITICAL STEP: Windows tracks line ends via CRLF, while Unix systems use LF [cite: 5]. Intercept discrepancies globally to prevent merge conflicts [cite: 5].	<pre># Windows Terminal Users: git config --global core.autocrlf true # Mac / Linux Terminal Users: git config --global core.autocrlf input</pre>

3. Generating & Registering Secure SSH Cryptographic Keys

SSH handles local-to-cloud handshakes via an asymmetric key pair: a private key that stays hidden on your machine, and a public key you upload to GitHub.

Deployment Stage	Instruction & Core Mechanics	Terminal Codes & Settings
1. Generate Key Pair	Run this in your terminal. Uses the highly secure, modern ed25519 algorithm. When prompted to "Enter file", hit Enter to accept the default slot. Press Enter twice again to bypass the passphrase for automation ease.	<pre>ssh-keygen -t ed25519 -C "email@example.com"</pre>
2. Copy Public Key (Windows)	Dump the raw text of the public key file directly into your clipboard memory cache. *Do not touch the private file without the .pub extension.*	<pre>cat ~/.ssh/id_ed25519.pub clip</pre>
3. Copy Public Key (Mac)	Equivalent clipboard capture command sequence tailored to macOS terminal pipelines.	<pre>pbcopy < ~/.ssh/id_ed25519.pub</pre>
4. Share Key to GitHub Cloud Portal	Log into GitHub in your browser. Navigating to the settings array allows you to paste the copied public string block.	1. Go to: Settings -> SSH and GPG keys 2. Click: New SSH Key 3. Paste your clipboard into the Key box, give it a title (e.g., "My Laptop"), and click Add SSH Key.
5. Verify Authenticated Link	Run an authentication test handshake directly from your terminal. If successful, you will see your GitHub profile user handle validated.	<pre>ssh -T git@github.com # Confirm 'yes' if asked about host authenticity</pre>

4. Core Git Operations & SSH Cloud Synchronization

Note: When creating repositories or copying sync links on GitHub, always choose the **SSH** tab instead of HTTPS to leverage your new key pair.

Git Operation	Instruction & Core Mechanics	Step-by-Step Code Loops
Initialize & Save	Convert an active project directory into a tracked local node, stage changes, and create an absolute history commit snapshot [cite: 5].	<pre>git init git add . git commit -m "Initialize secure project structure"</pre>
Bridge Cloud (SSH Link)	Connects your local machine repository directly to your remote target using the SSH format string [cite: 5].	<pre>git remote add origin git@github.com:username/repo.git</pre>
Push Payload	Enforce a default main workspace channel rename and securely push code up to GitHub via the SSH handshake loop [cite: 5].	<pre>git branch -M main git push -u origin main</pre>

